

FORTRAN — ΚΕΦΑΛΑΙΟ 3

ΔΟΜΕΣ ΕΠΑΝΑΛΗΨΗΣ

Σημειώσεις φοιτητή

Περιεχόμενα

- 3.1 Γιατί χρειαζόμαστε επανάληψη
- 3.2 Η δομή DO με μετρητή
- 3.3 Υπολογισμοί μέσα σε DO
- 3.4 Η δομή DO WHILE
- 3.5 DO με EXIT
- 3.6 CYCLE
- 3.7 Έλεγχος έναρξης και τερματισμού
- 3.8 Εμφωλευμένοι βρόχοι
- 3.9 Συνδυασμός IF + DO
- 3.10 Από εκφώνηση σε επανάληψη
- 3.11 Συχνά λάθη
- 3.12 Ασκήσεις εφαρμογής και έλεγχος γνώσεων

Στόχος του κεφαλαίου είναι η σωστή χρήση των δομών επανάληψης και η μετατροπή μιας εκφώνησης σε καθαρό αλγόριθμο Fortran.

3.1 Γιατί χρειαζόμαστε επανάληψη

1. Το πρόβλημα Έστω ότι θέλουμε να τυπώσουμε τους αριθμούς: Χωρίς επανάληψη, θα μπορούσαμε να γράψουμε:

```
program main
implicit none
print *, 1
print *, 2
print *, 3
print *, 4
print *, 5
end program main
```

 Αυτό δουλεύει. Όμως φαντάσου ότι θέλουμε να τυπώσουμε τους αριθμούς 1 έως 1000. Θα έπρεπε να

γράψουμε 1000 print . Αυτό είναι προφανώς κακή λύση. 2. Η βασική ιδέα της επανάληψης Η επανάληψη μάς επιτρέπει να πούμε: «Εκτέλεσε αυτό το τμήμα του προγράμματος ξανά και ξανά.» Για παράδειγμα:

```
do i = 1, 5
print *, i
end do
```

Εδώ το:

```
print *, i
```

δεν εκτελείται μία φορά. Εκτελείται 5 φορές. Το αποτέλεσμα είναι: Η Fortran διαθέτει διαφορετικές μορφές επανάληψης. Στο συγκεκριμένο μάθημα θα μας απασχολήσουν κυρίως:

```
DO με μετρητή
DO WHILE
DO με EXIT
CYCLE
```

εμφωλευμένοι βρόχοι πάνω σε αυτό θα στηριχθούν τα αθροίσματα, τα γινόμενα, το παραγοντικό και αργότερα οι πιο σύνθετες επαναλήψεις.

3.2 Η δομή DO με μετρητή

3.2.1 Η βασική σύνταξη

Η μορφή που δίνει το υλικό του καθηγητή είναι:

```
do αριθμητής = τιμή1, τιμή2 [, βήμα]
```

προτάσεις

```
end do
```

Για παράδειγμα:

```
integer :: i
do i = 1, 5
print *, i
end do
```

Εδώ: $i \rightarrow$ ο αριθμητής / μετρητής $1 \rightarrow$ αρχική τιμή $5 \rightarrow$ τελική τιμή το βήμα δεν γράφτηκε, άρα είναι 1. Επομένως ο i παίρνει

διαδοχικά:

3.2.2 Τι κάνει ο μετρητής;

Ας δούμε το:

```
do i = 1, 5
print *, i
end do
```

Σκέψου το σαν έναν μετρητή που κάνει:

```
i = 1
```

εκτέλεσε το σώμα

```
i = 2
```

εκτέλεσε το σώμα

```
i = 3
```

εκτέλεσε το σώμα

```
i = 4
```

εκτέλεσε το σώμα

```
i = 5
```

εκτέλεσε το σώμα. Μετά τελειώνει ο βρόχος. Άρα το:

```
print *, i
```

εκτελείται 5 φορές.

3.2.3 Αρχή, τέλος και βήμα

Το γενικό σχήμα:

```
do i = αρχή, τέλος, βήμα
```

σημαίνει: Ξεκίνα τον i από την αρχή, προχώρα με βήμα και επανάλαβε μέχρι να φτάσεις στο τέλος. Παράδειγμα 1 — βήμα 1

```
do i = 1, 5
print *, i
end do
```

Αποτελέσματα: Παράδειγμα 2 — βήμα 2

```
do i = 1, 10, 2
print *, i
end do
```

Ο i γίνεται: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7 \rightarrow 9$. Άρα: Το 10 δεν εμφανίζεται, γιατί η ακολουθία δεν φτάνει ακριβώς στο 10.

Παράδειγμα 3 — αντίστροφη μέτρηση. Μπορούμε να έχουμε αρνητικό βήμα:

```
do i = 5, 1, -1
print *, i
end do
```

Τότε: $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$ και η έξοδος: Το υλικό του καθηγητή περιγράφει το βήμα ως ακέραιο βήμα αύξησης ή μείωσης του αριθμητή. το υλικό της Fortran

3.2.4 Τι γίνεται όταν δεν γράψω το βήμα;

Αν γράψουμε:

```
do i = 1, 5
```

είναι σαν να γράψαμε:

```
do i = 1, 5, 1
```

Δηλαδή το βήμα είναι 1. Γι' αυτό το βήμα είναι προαιρετικό μόνο όταν είναι 1. το υλικό της Fortran

3.2.5 Πόσες φορές εκτελείται το DO ;

Αυτό είναι πολύ σημαντικό. Για:

```
do i = 1, 5
```

έχουμε: 1, 2, 3, 4, 5 άρα 5 επαναλήψεις. Για:

```
do i = 2, 10, 2
```

έχουμε: 2, 4, 6, 8, 10 άρα 5 επαναλήψεις. Για:

```
do i = 10, 2, -2
```

έχουμε: 10, 8, 6, 4, 2 άρα πάλι 5 επαναλήψεις.

3.2.6 Ο τύπος για τον αριθμό επαναλήψεων

Το υλικό του καθηγητή δίνει τον γενικό υπολογισμό: $2 - \text{τιμή τιμή } 1 + \text{βήμα } N = \max(\text{INT}(\frac{\text{ }{}}{\text{ }}, 0) \text{ βήμα}$ Δεν θέλω όμως να μάθεις αυτόν τον τύπο παπαγαλία. Για το επίπεδο που βρισκόμαστε, πολύ πιο χρήσιμο είναι να σκέφτεσαι: Ποιες τιμές θα πάρει πραγματικά ο μετρητής; Για παράδειγμα:

```
do i = 1, 10, 3
```

σκέψου: $1 \rightarrow 4 \rightarrow 7 \rightarrow 10$ Άρα έχουμε 4 επαναλήψεις.

3.2.7 Τι γίνεται αν τα όρια είναι «λάθος»;

Εδώ χρειάζεται προσοχή. Περίπτωση 1

```
do i = 5, 1
```

Έχουμε αρχή μεγαλύτερη από το τέλος, αλλά θετικό βήμα 1. Δεν μπορούμε να πάμε: $5 \rightarrow 6 \rightarrow 7 \rightarrow \dots$ και να φτάσουμε στο 1. Άρα ο βρόχος εκτελείται 0 φορές. Το υλικό το αναφέρει ρητά. το υλικό της Fortran Περίπτωση 2

```
do i = 1, 5, -1
```

Τώρα ξεκινάμε από 1, αλλά προσπαθούμε να κατέβουμε: $1 \rightarrow 0 \rightarrow -1 \rightarrow \dots$ ενώ θέλουμε να φτάσουμε στο 5. Και εδώ έχουμε 0 επαναλήψεις. το υλικό της Fortran Άρα ένας πολύ καλός κανόνας: Θετικό βήμα $\rightarrow$ πρέπει να κινούμαστε προς μεγαλύτερες τιμές. $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ Αρνητικό βήμα $\rightarrow$ πρέπει να κινούμαστε προς μικρότερες τιμές. $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$

3.2.8 Ένα ολοκληρωμένο πρόγραμμα

Ας γράψουμε ένα κανονικό πρόγραμμα:

```
program main
implicit none
integer :: i
do i = 1, 5
print *, i
end do
end program main
Τι πρέπει να προσέξεις; Πρώτα έχουμε τις δηλώσεις:
integer :: i
```

Μετά αρχίζει το εκτελέσιμο κομμάτι:

```
do i = 1, 5
do i = 1, 5
! κάνει έναν υπολογισμό
end do
```

Για παράδειγμα:

```
integer :: i, s
s = 0
do i = 1, 5
s = s + i
end do
print *, s
```

Εδώ ο βρόχος κάνει:

```
i = 1 → s = 1  
i = 2 → s = 3  
i = 3 → s = 6  
i = 4 → s = 10  
i = 5 → s = 15
```

Άρα:

```
s = 15
```

Και αυτό μας οδηγεί κατευθείαν στην 3.3 — Υπολογισμοί μέσα σε DO . Να κρατήσεις από το 3.2 Η βασική μορφή

```
do i = αρχή, τέλος, βήμα
```

...

```
end do
```

Αν δεν υπάρχει βήμα

```
do i = 1, 5
```

σημαίνει:

```
do i = 1, 5, 1
```

Αύξηση

```
do i = 1, 10, 2
```

→ 1, 3, 5, 7, 9 Μείωση

```
do i = 5, 1, -1
```

→ 5, 4, 3, 2, 1 Ασυμβατά όρια/βήμα

```
do i = 5, 1
```

→ 0 επαναλήψεις

```
do i = 1, 5, -1
```

→ 0 επαναλήψεις Και το σημαντικότερο: Το DO με μετρητή το χρησιμοποιούμε όταν έχουμε μια διαδικασία που πρέπει να επαναληφθεί για γνωστή ακολουθία τιμών. Αυτό ακριβώς θα αξιοποιήσουμε τώρα για αθροίσματα, γινόμενα και παραγοντικό, που είναι και η πρώτη άσκηση του Lab 5: υπολογισμός και εκτύπωση του αθροίσματος και του παραγοντικού των ακεραίων 1-10. Lab_5_presentation_2025_2026_st... Στο επόμενο 3.3 θα σταματήσουμε να «παίζουμε» με print και θα μάθουμε πώς ένα DO χτίζει έναν υπολογισμό βήμα-βήμα. Εκεί είναι που θα δέσει πραγματικά το κεφάλαιο.

αρχίζει να γίνεται πραγματικά χρήσιμο: δεν επαναλαμβάνουμε απλώς εντολές, αλλά χτίζουμε ένα αποτέλεσμα βήμα-βήμα.

3.3 Υπολογισμοί μέσα σε DO

3.3.1 Η βασική ιδέα

Έστω ότι θέλουμε να υπολογίσουμε: $1+2+3+4+5$ Μπορούμε φυσικά να γράψουμε:

```
s = 1 + 2 + 3 + 4 + 5
```

Αλλά αυτό δεν μας βοηθά όταν η εκφώνηση γίνει: $1 + 2 + 3 + \dots + 1000$ Εκεί θέλουμε έναν τρόπο να λέμε: «Πάρε κάθε αριθμό και πρόσθεσέ τον στο μέχρι τώρα άθροισμα.» Αυτό ακριβώς κάνει το DO .

3.3.2 Ο συσσωρευτής

Χρειαζόμαστε μια μεταβλητή που θα κρατάει το αποτέλεσμα μέχρι εκείνη τη στιγμή. Τη λέμε συσσωρευτή (accumulator). Για άθροισμα:

```
s = 0
do i = 1, 5
  s = s + i
end do
```

Η σημαντική εντολή είναι:

```
s = s + i
```

Μην τη διαβάζεις σαν μαθηματική εξίσωση. Στη Fortran σημαίνει: Υπολόγισε τη δεξιά πλευρά και αποθήκευσε το αποτέλεσμα στη μεταβλητή s . Άρα: παλιό $s + i \rightarrow$ νέο s

3.3.3 Ας παρακολουθήσουμε την εκτέλεση με το χέρι

Έχουμε:

```
s = 0
do i = 1, 5
  s = s + i
end do
```

1η επανάληψη

```
i = 1
s = 0
```

Άρα:

```
s = 0 + 1 = 1
```

Τώρα:

```
s = 1
```

2η επανάληψη και:

```
do i = 1, 5
  p = p * i
end do
```

Η εξέλιξη είναι:

```
i = 1 → p = 1 × 1 = 1
i = 2 → p = 1 × 2 = 2
i = 3 → p = 2 × 3 = 6
i = 4 → p = 6 × 4 = 24
i = 5 → p = 24 × 5 = 120
```

Άρα: $1 \times 2 \times 3 \times 4 \times 5 = 120$

3.3.6 Ο γενικός κανόνας

Έχουμε λοιπόν: Για άθροισμα

```
s = 0
do i = ...
  s = s + ...
end do
```

Για γινόμενο

```
p = 1
do i = ...
  p = p * ...
end do
```

Αυτό το μοτίβο θα το συναντάμε πάρα πολύ συχνά.

3.3.7 Παραγοντικό

Το παραγοντικό ενός θετικού ακεραίου n είναι: $n! = 1 \times 2 \times 3 \times \dots \times n$ Για παράδειγμα: $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$ Άρα το παραγοντικό είναι στην πραγματικότητα ένα γινόμενο που χτίζεται μέσα σε DO . Για $n = 5$:

```
p = 1
do i = 1, 5
  p = p * i
end do
```

Στο υλικό του καθηγητή παρουσιάζεται ακριβώς αυτή η λογική για τον υπολογισμό του παραγοντικού, μαζί με ισοδύναμες μορφές DO , DO WHILE και DO με EXIT .

3.3.8 Άθροισμα και παραγοντικό μαζί

Και τώρα μπορούμε να συνδυάσουμε τα δύο. Για τους αριθμούς 1-10 θέλουμε: το άθροισμα το γινόμενο, δηλαδή το 10! Το Lab 5 ζητά ακριβώς πρόγραμμα που υπολογίζει και εκτυπώνει το άθροισμα και το παραγοντικό των ακεραίων 1-10. Lab_5_presentation_2025_2026_st... Μια απλή λύση είναι:

```
program main
implicit none
integer :: i
integer :: s, p
s = 0
p = 1
do i = 1, 10
  s = s + i
  p = p * i
end do
print *, "Sum =", s
print *, "Factorial =", p
end program main Στο τέλος:
Sum = 55
Factorial = 3628800
```

3.3.9 Πρόσεξε κάτι πολύ σημαντικό

Μέσα στον ίδιο βρόχο έχουμε:

```
s = s + i
p = p * i
```

Δηλαδή σε κάθε επανάληψη ενημερώνονται και τα δύο αποτελέσματα. Για $i = 3$, για παράδειγμα:

```
s = s + 3
p = p * 3
```

Άρα το DO μπορεί να κάνει πολλούς υπολογισμούς ταυτόχρονα, αρκεί να έχουν την ίδια επαναληπτική λογική.

3.3.10 Ενδιάμεσα αποτελέσματα

Το Lab 5 δεν σταματά μόνο στο τελικό αποτέλεσμα. Η Άσκηση 2 ζητά να υπολογίσουμε τη σειρά:

```
S = 5 + 3 - 9 + 27 - 81 + ...
```

και να εκτυπώσουμε τόσο τις ενδιάμεσες όσο και την τελική τιμή του S . Lab_5_presentation_2025_2026_st... Αυτό μας δείχνει μια ακόμη σημαντική χρήση του συσσωρευτή. Για παράδειγμα, αν έχουμε:

```
s = 0
do i = 1, 5
  s = s + i
  print *, s
end do
```

εξετάζεται: Δεν βλέπουμε μόνο το τελικό 15 . Βλέπουμε πώς δημιουργήθηκε: $1 \rightarrow 3 \rightarrow 6 \rightarrow 10 \rightarrow 15$ Αυτό είναι πολύ χρήσιμο όταν θέλουμε να ελέγξουμε αν ο αλγόριθμός μας δουλεύει σωστά.

3.3.11 Ένα πολύ συχνό λάθος

Λάθος αρχικοποίηση Για άθροισμα: Και αν δεις:

```
p = 1
do i = 2, 5
  p = p * i
end do
```

τότε:

```
p = 2
p = 6
p = 24
p = 120
```

Άρα:

$p = 120$

Στο DO με μετρητή ξέρουμε την ακολουθία των επαναλήψεων. Στο DO WHILE η επανάληψη εξαρτάται από μια συνθήκη. Αυτό είναι και το επόμενο βήμα που προβλέπει το roadmap μας.

3.4 Η δομή DO WHILE

3.4.1 Η βασική ιδέα

Θυμήσου από το Κεφάλαιο 2: Μια συνθήκη είναι μια ερώτηση που έχει απάντηση: TRUE ή FALSE Το DO WHILE χρησιμοποιεί ακριβώς αυτή την ιδέα. Σημαίνει: «Επανάλαβε όσο η συνθήκη είναι TRUE.» Η βασική μορφή είναι:

```
do while (συνθήκη)
```

προτάσεις

```
end do
```

Για παράδειγμα:

```
integer :: i
i = 1
do while (i <= 5)
  print *, i
  i = i + 1
end do
```

3.4.2 Πώς λειτουργεί;

Ας παρακολουθήσουμε την εκτέλεση βήμα-βήμα. Έχουμε:

```
i = 1
do while (i <= 5)
  print *, i
  i = i + 1
end do
```

Πριν ξεκινήσει ο βρόχος

```
i = 1
```

Ελέγχουμε: $i \leq 5$ $1 \leq 5$ TRUE Άρα μπαίνουμε στον βρόχο. Τυπώνεται:

```
i = i + 1
```

οπότε:

```
i = 2
```

Επόμενη επανάληψη Ελέγχουμε ξανά: $2 \leq 5$ TRUE Τυπώνεται: και:

```
i = 3
```

Το ίδιο συνεχίζεται:

```
i = 3 → TRUE → print 3 → i = 4
i = 4 → TRUE → print 4 → i = 5
i = 5 → TRUE → print 5 → i = 6
```

Τώρα:

```
i = 6
```

και ελέγχουμε: $6 \leq 5$ FALSE Άρα σταματάμε.

3.4.3 Το κρίσιμο σημείο

Σε ένα DO WHILE η συνθήκη ελέγχεται πριν από κάθε επανάληψη. Σχηματικά: ↓ έλεγχος συνθήκης ↓ TRUE; ↓ εκτέλεση σώματος ↓ ξανά έλεγχος ενώ: FALSE ↓ έξοδος Άρα: Αν η συνθήκη είναι FALSE από την αρχή, το σώμα του DO WHILE δεν εκτελείται ούτε μία φορά. Αυτό είναι σημαντική διαφορά από μια λογική «κάνε πρώτα και μετά έλεγξε».

3.4.4 DO με μετρητή vs DO WHILE

Ας συγκρίνουμε: DO

```
do i = 1, 5
  print *, i
end do
```

Εδώ λέμε: Για τις τιμές 1 έως 5, εκτέλεσε το σώμα.

```
DO WHILE
i = 1
do while (i <= 5)
  print *, i
  i = i + 1
end do
```

Εδώ λέμε: Όσο $i \leq 5$, εκτέλεσε το σώμα. Το αποτέλεσμα μπορεί να είναι ίδιο, αλλά η λογική είναι διαφορετική.

3.4.5 Πότε προτιμάμε DO WHILE ;

Το DO με μετρητή είναι φυσικό όταν έχουμε: «Για $i = 1$ έως $N \dots$ » Το DO WHILE είναι φυσικό όταν η εκφώνηση λέει: «Επανάλαβε όσο...» ή: «Συνέχισε μέχρι να...» ή: «Όσο η ποσότητα είναι μικρότερη από...» Για παράδειγμα: «Διπλασίαζε ένα ποσό όσο αυτό παραμένει μικρότερο ή ίσο από ένα όριο.» Εδώ δεν μας ενδιαφέρει εξαρχής να πούμε: κάνε 7 επαναλήψεις. Γιατί δεν ξέρουμε απαραίτητα πόσες θα χρειαστούν. Μας ενδιαφέρει η συνθήκη. Αυτό συνδέεται άμεσα με την Άσκηση 3 του Lab 5, όπου ζητείται να ξεκινήσει ένα ποσό από 10, να διπλασιάζεται και να σταματήσει όταν ξεπεράσει το 1000.

3.4.6 Παράδειγμα: ποσό που διπλασιάζεται

Ας δούμε τη λογική χωρίς να την κάνουμε ακόμη σύνθετη. Το υλικό του καθηγητή το τονίζει ξεκάθαρα: στις επαναλήψεις υπό συνθήκη πρέπει να εξασφαλίζουμε και ότι η επανάληψη ξεκινά και ότι η επανάληψη τελειώνει, άρα η μεταβλητή που συμμετέχει στη συνθήκη πρέπει να αλλάζει μέσα στην επανάληψη.

3.4.8 Το επικίνδυνο λάθος: άπειρος βρόχος

Κοίτα:

```
i = 1
do while (i <= 5)
  print *, i
end do
```

Τι γίνεται εδώ; Η συνθήκη είναι: $i \leq 5$ και:

```
i = 1
```

Άρα είναι TRUE. Μπαίνουμε στον βρόχο. Τυπώνουμε 1. Ξαναγυρίζουμε στην αρχή. Το i όμως παραμένει 1. Άρα: $1 \leq 5 \rightarrow \text{TRUE}$ ξανά. Και ξανά. Και ξανά... Το πρόγραμμα μπορεί να συνεχίσει επ' άπειρον. Αυτό είναι ο άπειρος βρόχος (infinite loop).

3.4.9 Πώς τον αποφεύγουμε;

Πρέπει να υπάρχει μέσα στον βρόχο κάποια ενέργεια που μπορεί να κάνει τη συνθήκη τελικά FALSE. Σωστό:

```
i = 1
do while (i <= 5)
  print *, i
  i = i + 1
end do
```

Λάθος:

```
i = 1
do while (i <= 5)
  print *, i
end do
```

Ο κανόνας του roadmap μας είναι ακριβώς αυτός: Η μεταβλητή της συνθήκης πρέπει να μπορεί να αλλάξει μέσα στον βρόχο.

3.4.10 Και η αρχική τιμή είναι εξίσου σημαντική

Δες: «Υπολόγισε το άθροισμα από 1 έως 100.»

```
do i = 1, 100
  s = s + i
end do
```

Άγνωστο πλήθος επαναλήψεων «Διπλασίαζε το ποσό όσο δεν έχει ξεπεράσει το όριο.»

```
do while (amount <= limit)
  amount = amount * 2.0
end do
```

Στο πρώτο ξέρουμε το εύρος. Στο δεύτερο η συνθήκη καθορίζει πόσο θα διαρκέσει η επανάληψη. Αυτή η διάκριση θα μας βοηθήσει πολύ αργότερα στην ενότητα 3.10 — Από εκφώνηση σε επανάληψη.

3.4.14 DO WHILE — πλήρες παράδειγμα

Ας το γράψουμε ως ολοκληρωμένο πρόγραμμα:

```
program main
  implicit none
  integer :: i
  i = 1
  do while (i <= 5)
    print *, i
    i = i + 1
  end do
end program main
```

 Έξοδος: Η λογική είναι:

```

i = 1
↓ i <= 5 ? ↓ TRUE
print i
↓
i = i + 1
↓ ξανά έλεγχος ↓ ... ↓
i = 6
↓ 6 <= 5 ? ↓ FALSE τέλος

```

3.4.15 Μία λεπτή αλλά σημαντική παρατήρηση

Στο DO με μετρητή:

```

do i = 1, 5
...
end do

```

η Fortran χειρίζεται την εξέλιξη του i . Ξεκινήσουμε έναν βρόχο χωρίς να δηλώσουμε από πριν πότε ακριβώς θα τελειώσει και να αποφασίσουμε μέσα στον βρόχο πότε θα φύγουμε. Το DO με EXIT παρουσιάζεται στο υλικό του καθηγητή ως «επανάλαβε μέχρις ότου», ενώ το roadmap μας το τοποθετεί ακριβώς εδώ, μετά το DO WHILE.

3.5 DO με EXIT

3.5.1 Η βασική σύνταξη

Η μορφή είναι: do προτάσεις

```
if (συνθήκη) exit
end do
```

Δηλαδή: 1. ξεκινάμε έναν γενικό DO 2. εκτελούμε τις εντολές 3. ελέγχουμε μια συνθήκη 4. αν η συνθήκη είναι TRUE, εκτελούμε EXIT 5. βγαίνουμε από τον βρόχο Το υλικό του καθηγητή περιγράφει ακριβώς αυτή τη λογική: όταν ισχύσει η συνθήκη του

EXIT, ο έλεγχος μεταφέρεται αμέσως μετά το END DO, το υλικό της Fortran

3.5.2 Τι είναι το EXIT ;

Το:

```
exit
```

σημαίνει: «Βγες τώρα από τον βρόχο.» Δεν σημαίνει: «πήγαινε στην επόμενη επανάληψη». Αυτό θα το κάνουμε με το CYCLE, που θα μάθουμε στην επόμενη ενότητα. Το EXIT σημαίνει οριστική έξοδος από τον συγκεκριμένο βρόχο.

3.5.3 Το πρώτο απλό παράδειγμα

Θέλουμε να τυπώσουμε: και μετά να σταματήσουμε. Μπορούμε να γράψουμε:

```
program main
implicit none
integer :: i
i = 1
```

do

```
print *, i
if (i >= 5) exit
i = i + 1
end do
end program main Ας δούμε τι συμβαίνει.
PRINT "After loop"
```

Δηλαδή το EXIT δεν σταματά ολόκληρο το πρόγραμμα. Σταματά τον βρόχο. Αυτό είναι πολύ σημαντικό.

3.5.6 DO WHILE και DO ... EXIT

Μπορούμε να πετύχουμε παρόμοια αποτελέσματα με δύο διαφορετικούς τρόπους.

```
DO WHILE
i = 1
do while (i <= 5)
print *, i
i = i + 1
end do
DO ... EXIT
i = 1
```

do

```
print *, i
i = i + 1
if (i > 5) exit
end do
```

Και τα δύο μπορούν να τερματίσουν όταν το i ξεπεράσει το 5. Όμως η λογική τους είναι διαφορετική.

3.5.7 Ποια είναι η διαφορά στη σκέψη;

```
DO WHILE
```

Η συνθήκη βρίσκεται στην είσοδο: ↓ έλεγχος ↓ TRUE? ✓ ✗ ΝΑΙ ΟΧΙ ↓ ↓ σώμα τέλος ↓ ξανά έλεγχος

```
DO ... EXIT
```

Ο βρόχος ξεκινά: ↓ εκτέλεση ↓ έλεγχος EXIT ↓ TRUE? ✓ ✗ ΝΑΙ ΟΧΙ ↓ ↓

EXIT συνέχεια

↓ ξανά

```
if (x == 0) exit
print *, x
end do
```

Αν $x = 0$, το print δεν εκτελείται. Αντίθετα, αν γράψαμε: do

```
read *, x
print *, x
if (x == 0) exit
end do
```

τότε το 0 θα τυπωνόταν πριν γίνει η έξοδος. Άρα πρέπει πάντα να αναρωτιόμαστε: Θέλω να γίνει η συγκεκριμένη εντολή πριν ή μετά τον έλεγχο τερματισμού; Αυτό θα γίνει πολύ σημαντικό όταν λύνουμε ασκήσεις.

3.5.11 Το EXIT μπορεί να χρησιμοποιηθεί με IF

Όπως έχουμε ήδη μάθει από το Κεφάλαιο 2:

```
if (συνθήκη) then
```

... end if μπορούμε να έχουμε: do

```
if (x > 100) then
exit
end if
end do
```

ή τη σύντομη μορφή: do

```
if (x > 100) exit
```

Η δεύτερη μορφή είναι πολύ συνηθισμένη στο υλικό του καθηγητή. το υλικό της Fortran

3.5.12 Το EXIT δεν χρειάζεται να είναι στο τέλος

Μπορεί να βρίσκεται στη μέση του σώματος. Για παράδειγμα: do εντολές_1

```
if (συνθήκη) exit
```

εντολές_2

```
end do
```

Αν η συνθήκη γίνει TRUE : εντολές_1 ↓

```
EXIT
```

↓ τέλος βρόχου Οι εντολές_2 δεν εκτελούνται σε εκείνη την επανάληψη. Το υλικό του καθηγητή το περιγράφει ως δύο τμήματα, πριν και μετά την ενδιάμεση συνθήκη EXIT : όσο η συνθήκη δεν ισχύει, εκτελούνται και τα δύο· όταν ισχύσει, ο έλεγχος φεύγει αμέσως από το DO . το υλικό της Fortran

3.5.13 Ονομασμένος βρόχος

Επειδή αργότερα θα έχουμε εμφωλευμένα DO , υπάρχει και η δυνατότητα να δώσουμε όνομα στον βρόχο: outer: do ...

```
if (condition) exit outer
end do outer
```

Αυτό σημαίνει: «Βγες από τον συγκεκριμένο βρόχο outer .» Στο υλικό του καθηγητή εμφανίζεται ακριβώς αυτή η σύνταξη με ονομασμένες δομές. Για την ώρα δεν χρειάζεται να το χρησιμοποιούμε στα απλά παραδείγματα. Θα το ξαναδούμε στους εμφωλευμένους βρόχους.

3.5.14 Ένα πολύ ωραίο παράδειγμα του καθηγητή

Στο το υλικό της Fortran υπάρχει και παράδειγμα όπου χρησιμοποιείται EXIT για να βρεθεί μια ρίζα μιας εξίσωσης. Η λογική είναι περίπου:

```
do while (x < xb)
x = x + step
y = x**3 - x**2 - 2
if (abs(y) < 1.0e-6) exit
end do
```

Εδώ έχουμε έναν πολύ ωραίο συνδυασμό: επανάληψη υπολογισμό IF

```
EXIT
```

κριτήριο τερματισμού Δεν ψάχνουμε πόσες φορές πρέπει να εκτελέσουμε τον βρόχο. Ψάχνουμε πότε θα ικανοποιηθεί το κριτήριο: $\text{abs}(y) < 1.0\text{e-}6$ Αυτό είναι ακριβώς το είδος προβλήματος όπου το EXIT γίνεται πολύ φυσικό.

3.5.15 EXIT ≠ CYCLE

Αυτό θέλω να το ξεχωρίσουμε από τώρα.

```
EXIT
if (condition) exit
```

σημαίνει: Τέλος ολόκληρου του βρόχου.

```
CYCLE
if (condition) cycle
```

σημαίνει: Τέλος μόνο της τρέχουσας επανάληψης — πήγαινε στην επόμενη. Παράδειγμα: Αν στο 3 κάνουμε EXIT , παίρνουμε: και τελειώνει ο βρόχος. Αν στο 3 κάνουμε CYCLE , τότε: και ο βρόχος συνεχίζει. Το CYCLE είναι η επόμενη ενότητα, οπότε θα το δούμε αναλυτικά εκεί.

3.5.16 Το βασικό μοτίβο που χρειάζεται να μπορείς

Όταν βλέπεις μια εκφώνηση: «Επανάλαβε μέχρι να συμβεί κάτι συγκεκριμένο.» σκέψου: do

```
! κάνε τη δουλειά
if (συνθήκη_τερματισμού) exit
end do
```

Για παράδειγμα: «Διάβαζε αριθμούς μέχρι να δοθεί το 0.» do

```
read *, x
if (x == 0) exit
! επεξεργασία του x
end do
```

Συχνό λάθος: EXIT χωρίς βρόχο Το:

```
if (x == 0) exit
```

έχει νόημα μέσα σε βρόχο. Δεν γράφουμε:

```
program main
implicit none
integer :: x
read *, x
if (x == 0) exit ! end program main
```

Το EXIT αφορά έξοδο από δομή επανάληψης. Τι κρατάμε από το 3.5 Βασική μορφή do ...

```
if (συνθήκη) exit
end do
```

Το EXIT σημαίνει: Βγες αμέσως από τον τρέχοντα βρόχο. Όταν η συνθήκη γίνει .TRUE. , εκτελείται το CYCLE . Ο βρόχος δεν τελειώνει. Απλώς παραλείπεται το υπόλοιπο σώμα της συγκεκριμένης επανάληψης και ξεκινά η επόμενη.

3.6.2 EXIT vs CYCLE

Αυτό θέλω να το ξεκαθαρίσουμε απόλυτα.

```
EXIT
do i = 1, 5
if (i == 3) exit
print *, i
end do
```

Όταν i = 3 : και τέλος βρόχου.

```
CYCLE
do i = 1, 5
if (i == 3) cycle
print *, i
end do
```

Όταν i = 3 , κάνουμε CYCLE . Άρα παραλείπουμε το:

```
print *, i
```

αλλά ο βρόχος συνεχίζει. Έξοδος: Χρήσιμος κανόνας:

```
EXIT
```

↓ Φεύγω από τον βρόχο.

```
CYCLE
```

↓ Φεύγω μόνο από την τρέχουσα επανάληψη.

3.6.3 Τι σημαίνει «τρέχουσα επανάληψη»;

Έστω ότι έχουμε:

```
do i = 1, 5
print *, "A"
if (i == 3) cycle
```

```
print *, "B"
end do
```

Για $i = 1$: A B Για $i = 2$: A B σημαίνει: ο i είναι άρτιος. Εδώ βλέπουμε πολύ καθαρά τη σύνδεση του Κεφαλαίου 2 (IF) με το Κεφάλαιο 3 (DO).

3.6.6 Ένα πιο ρεαλιστικό παράδειγμα

Έστω ότι θέλουμε να υπολογίσουμε το άθροισμα των θετικών αριθμών από μια σειρά τιμών και να αγνοούμε τους αρνητικούς. Με CYCLE μπορούμε να γράψουμε:

```
do i = 1, n
  if (a(i) < 0) cycle
  s = s + a(i)
end do
```

Η λογική είναι: πάρε $a(i)$ ↓ είναι αρνητικό; ✓ ✗ ΝΑΙ ΟΧΙ ↓ ↓

CYCLE πρόσθεσέ το

↓ ↓ επόμενο επόμενος Άρα το CYCLE μας επιτρέπει να απομακρύνουμε γρήγορα τις περιπτώσεις που δεν μας ενδιαφέρουν.

3.6.7 CYCLE μέσα σε DO WHILE

Το CYCLE δεν χρησιμοποιείται μόνο σε DO με μετρητή. Μπορεί να χρησιμοποιηθεί και σε:

```
do while (condition)
  if (κάποια_άλλη_συνθήκη) cycle
...
end do
```

Παράδειγμα:

```
i = 0
do while (i < 10)
  i = i + 1
  if (i == 5) cycle
  print *, i
end do
```

Θα πάρουμε: Το 5 παραλείφθηκε, αλλά ο βρόχος συνέχισε.

3.6.8 Προσοχή στο DO WHILE + CYCLE

Εδώ υπάρχει μια λεπτομέρεια που μπορεί να δημιουργήσει άπειρο βρόχο. Δες:

```
i = 1
end do
```

Για $i = 3$ δεν εκτελείται ούτε:

```
print *, i
```

ούτε:

```
print *, i**2
```

Γιατί το CYCLE βρίσκεται πριν από αυτά. Αντίθετα:

```
do i = 1, 5
  print *, i
  if (i == 3) cycle
  print *, i**2
end do
```

Για $i = 3$:

```
print 3
CYCLE
```

Άρα το 3 τυπώνεται, αλλά το 9 όχι.

3.6.11 IF ... CYCLE

Η πιο συνηθισμένη μορφή που θα βλέπεις είναι:

```
if (condition) cycle
```

Αυτό είναι ιδιαίτερα βολικό όταν θέλουμε να πούμε: «Αν αυτή η περίπτωση δεν μας ενδιαφέρει, προσπέρασέ την.» Το υλικό του το υλικό της Fortran περιέχει πραγματικό παράδειγμα με:

```
if (i == 0) cycle
```

μέσα σε DO , όπου η συγκεκριμένη περίπτωση παραλείπεται και ο υπολογισμός συνεχίζεται για τις υπόλοιπες.

3.6.12 Ονομασμένο CYCLE

Όπως και με το EXIT , μπορούμε να έχουμε ονομασμένο βρόχο: outer: do i = 1, 10

```
if (condition) cycle outer
```

...

```
end do outer
```

Αυτό αποκτά ιδιαίτερη σημασία όταν έχουμε βρόχο μέσα σε βρόχο. Για παράδειγμα: outer: do i = 1, 3

```
do j = 1, 3
  if (condition) cycle outer
end do
end do outer
```

Εδώ το:

```
cycle outer
```

δεν αφορά τον εσωτερικό DO . Πηγαίνει στην επόμενη επανάληψη του outer . Αυτό θα γίνει πολύ πιο κατανοητό στην 3.8 — Εμφωλευμένοι βρόχοι, οπότε προς το παρόν απλώς κράτα ότι υπάρχει αυτή η δυνατότητα.

3.6.13 CYCLE vs IF ... ELSE

Πολλές φορές μπορούμε να γράψουμε τον ίδιο αλγόριθμο και χωρίς CYCLE . Για παράδειγμα: Με CYCLE

```
do i = 1, 10
  if (mod(i,2) == 0) cycle
  print *, i
end do
```

Χωρίς CYCLE

```
do i = 1, 10
  if (mod(i,2) /= 0) then
    print *, i
  end if
end do
```

Και τα δύο τυπώνουν: Τότε γιατί να χρησιμοποιήσουμε CYCLE ; Επειδή σε μεγαλύτερο σώμα βρόχου μπορεί να κάνει τη λογική πολύ πιο καθαρή:

```
do i = 1, n
  if (μη_έγκυρη_περίπτωση) cycle
  ! εδώ ξέρουμε ότι η περίπτωση είναι έγκυρη
  ! και συνεχίζουμε τον βασικό υπολογισμό
end do
```

Αυτό συχνά είναι πιο ευανάγνωστο από ένα τεράστιο:

```
if (έγκυρη_περίπτωση) then
```

... .. end if

3.6.14 Ένα παράδειγμα που αξίζει να μάθεις

Θέλουμε να διαβάσουμε 10 αριθμούς και να υπολογίσουμε το άθροισμα μόνο των θετικών.

```
integer :: i, x, s
s = 0
do i = 1, 10
  read *, x
  if (x <= 0) cycle
  s = s + x
end do
print *, "Sum =", s
```

Η λογική είναι εξαιρετικά καθαρή: Μικρός έλεγχος Παράδειγμα:

```
do i = 1, 6
  if (i == 2) cycle
  if (i == 5) exit
  print *, i
end do
```

Ας το τρέξουμε νοητά:

```
i = 1
i == 2 → FALSE
i == 5 → FALSE
print → 1
i = 2
```



```
i == 2 → TRUE  
CYCLE
```

δεν φτάνουμε ποτέ στο print

```
i = 3
```

→ print 3

```
i = 4
```

→ print 4

```
i = 5  
i == 2 → FALSE  
i == 5 → TRUE  
EXIT
```

Ο βρόχος τελειώνει. Άρα: Αυτό είναι ακριβώς το επίπεδο κατανόησης που θέλω: να μπορείς να παρακολουθείς τη ροή μιας επανάληψης και να ξέρεις ποια εντολή εκτελείται και ποια παραλείπεται. Στο επόμενο πάμε στο 3.7 — Έλεγχος έναρξης και τερματισμού, όπου θα μαζέψουμε όλα αυτά και θα μάθουμε να εντοπίζουμε τα επικίνδυνα σημεία: λάθος αρχικοποίηση, συνθήκη που δεν αλλάζει, λάθος όρια και άπειρους βρόχους. Αυτή η ενότητα είναι μικρή σε σύνταξη, αλλά πολύ σημαντική στον τρόπο που σκέφτεσαι έναν βρόχο. Το βασικό μήνυμά του

υλικού είναι ότι σε επαναλήψεις υπό συνθήκη πρέπει να εξασφαλίζουμε δύο πράγματα: να μπορεί η επανάληψη να ξεκινήσει και να μπορεί κάποτε να τελειώσει. το υλικό της Fortran

3.7 Έλεγχος έναρξης και τερματισμού

3.7.1 Ο βρόχος δεν είναι απλώς «γράψε ένα DO»

Μέχρι τώρα έχουμε δει:

```
do i = 1, 10
```

...

```
end do
```

και:

```
do while (condition)
```

...

```
end do
```

και: do ...

```
if (condition) exit  
end do
```

Σε όλα αυτά πρέπει να μπορούμε να απαντήσουμε: 1. Πότε ξεκινάει η επανάληψη; 2. Πότε σταματάει; 3. Τι αλλάζει από τη μία επανάληψη στην επόμενη; Αν δεν μπορούμε να απαντήσουμε σε αυτά, υπάρχει μεγάλη πιθανότητα να έχουμε λάθος αλγόριθμο.

3.7.2 Οι δύο βασικές προϋποθέσεις

Το υλικό του καθηγητή το διατυπώνει πολύ συγκεκριμένα: Στις δομές DO υπό συνθήκη πρέπει να διασφαλίσουμε:

α) ότι η επανάληψη ξεκινάει β) ότι η επανάληψη τελειώνει. το υλικό της Fortran Ας τα δούμε ξεχωριστά.

3.7.3 Πρέπει να μπορεί να ξεκινήσει

Έχουμε:

```
i = 1  
do while (i <= 5)  
  print *, i  
  i = i + 1  
end do
```

Πριν μπούμε στον βρόχο:

```
i = 1
```

Ελέγχουμε: $1 \leq 5 \rightarrow .TRUE$. Άρα ο βρόχος ξεκινάει. Αν όμως έχουμε:

```
i = 10  
do while (i <= 5)  
  print *, i  
  i = i + 1  
end do
```

η αρχική συνθήκη είναι: $10 \leq 5 \rightarrow .FALSE$. Άρα: 0 επαναλήψεις Αυτό δεν είναι απαραίτητα λάθος. Είναι όμως κάτι που πρέπει να το έχεις προβλέψει.

3.7.4 Πρέπει να μπορεί να τελειώσει

Τώρα το πιο επικίνδυνο:

```
i = 1  
do while (i <= 5)  
  print *, i  
end do
```

Το κρίσιμο βήμα είναι: «αλλάζω κάτι» Αν τίποτα σχετικό με τη συνθήκη δεν αλλάζει, πρέπει να αναρωτηθείς: Πώς ακριβώς θα γίνει ποτέ FALSE;

3.7.7 Παράδειγμα: σωστός βρόχος

```
i = 1  
do while (i < 100)  
  i = i * 2  
end do
```

Εδώ: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$ Όταν φτάσουμε: $128 < 100 \rightarrow .FALSE$. Άρα σταματάμε.

3.7.8 Παράδειγμα: προβληματικός βρόχος

```
i = 1
do while (i < 100)
i = i - 1
end do
```

Τι συμβαίνει; $1 \rightarrow 0 \rightarrow -1 \rightarrow -2 \rightarrow -3 \rightarrow \dots$ Η συνθήκη: $i < 100$ παραμένει πάντα .TRUE. . Άρα έχουμε άπειρη επανάληψη. Αυτό είναι ένα πολύ σημαντικό μάθημα: Δεν αρκεί να αλλάζει η μεταβλητή. Πρέπει να αλλάζει προς την κατεύθυνση που οδηγεί στον τερματισμό.

3.7.9 Άλλο παράδειγμα: λάθος κατεύθυνση

```
i = 10
do while (i > 0)
i = i + 1
end do
```

Εδώ: $10 \rightarrow 11 \rightarrow 12 \rightarrow 13 \rightarrow \dots$ Η συνθήκη: $i > 0$ δεν πρόκειται να γίνει FALSE. Άρα πάλι άπειρος βρόχος. Για να τερματίσει, θα

έπρεπε να έχουμε:

```
i = i - 1
```

οπότε: $10 \rightarrow 9 \rightarrow 8 \rightarrow \dots \rightarrow 1 \rightarrow 0$ και: $0 > 0 \rightarrow$.FALSE.

3.7.10 Το ίδιο πρόβλημα υπάρχει με EXIT

Δεν πρέπει να θεωρούμε ότι το EXIT λύνει αυτόματα όλα τα προβλήματα. Για παράδειγμα: do

```
print *, "Hello"
if (x > 10) exit
end do
```

Πώς θα γίνει: $x > 10$ αν το x δεν αλλάζει ποτέ; Αν ξεκινάμε με: $x = 5$ θα έχουμε: $5 > 10 \rightarrow$ FALSE $5 > 10 \rightarrow$ FALSE $5 > 10 \rightarrow$ FALSE ... Άρα και εδώ μπορούμε να έχουμε άπειρο βρόχο. Το EXIT είναι μηχανισμός τερματισμού· εμείς πρέπει να εξασφαλίσουμε ότι κάποτε θα ενεργοποιηθεί.

3.7.11 CYCLE θέλει ακόμη περισσότερη προσοχή

Θυμήσου το παράδειγμα που είδαμε:

```
i = 1
do while (i <= 5)
if (i == 3) cycle
i = i + 1
end do
```

Όταν:

```
i = 3
```

γίνεται:

```
cycle
```

και δεν εκτελείται:

```
i = i + 1
```

Άρα:

```
i = 3
```

για πάντα. μπορούμε να πούμε: i ξεκινά από 1 i αυξάνεται κατά 1 το όριο είναι 10 άρα τελικά $i = 11$ και η συνθήκη γίνεται FALSE Άρα τερματίζει. Αυτός ο τρόπος σκέψης είναι πολύ πιο σημαντικός από το να απομνημονεύσεις απλώς τη σύνταξη.

3.7.14 Γνωστό πλήθος vs συνθήκη

Ας δούμε ξανά τη διαφορά.

```
DO με μετρητή
do i = 1, 10
```

...

```
end do
```

Εδώ η Fortran ξέρει το εύρος του μετρητή. Δεν χρειάζεται να γράψουμε εμείς:

```
i = i + 1
```

Η εξέλιξη του μετρητή είναι μέρος της δομής.

```
DO WHILE
i = 1
do while (i <= 10)
```

...

```
i = i + 1
end do
```

Εδώ εμείς πρέπει να εξασφαλίσουμε την εξέλιξη που θα οδηγήσει στον τερματισμό. Αυτός είναι ένας από τους λόγους που το

DO WHILE και το DO ... EXIT χρειάζονται μεγαλύτερη προσοχή.

3.7.15 Σύνδεση με τις ασκήσεις

Αυτό δεν είναι θεωρία «στον αέρα». Στις Ασκήσεις 01, για παράδειγμα, υπάρχει άσκηση όπου ζητείται να διαβάζεται μια σειρά θετικών πραγματικών αριθμών χωρίς γνωστό εκ των προτέρων πλήθος, και η είσοδος σταματά όταν ο χρήστης δώσει αρνητικό αριθμό. Ασκήσεις_01 Εδώ πρέπει να σκεφτούμε: Δεν ξέρω πόσες τιμές θα διαβαστούν. ↓ Δεν είναι φυσικό ένα DO 1..N. ↓ Χρειάζομαι συνθήκη τερματισμού. ↓

DO WHILE ή DO + EXIT.

Και αμέσως μετά πρέπει να αναρωτηθούμε: Πότε θα γίνει η συνθήκη FALSE / πότε θα ενεργοποιηθεί το EXIT ; Αυτό ακριβώς είναι το νόημα της 3.7.

3.7.16 Ένα πολύ χρήσιμο παράδειγμα

Έστω:

```
do j = 1, 4
```

...

```
end do
end do
```

Έχουμε: έναν εξωτερικό βρόχο με i έναν εσωτερικό βρόχο με j Ο εσωτερικός βρόχος βρίσκεται μέσα στον εξωτερικό.

3.8.2 Πώς εκτελείται;

Αυτό είναι το σημαντικότερο σημείο. Έχουμε:

```
do i = 1, 3
do j = 1, 4
print *, i, j
end do
end do
```

Πολλοί στην αρχή σκέφτονται ότι οι δύο μετρητές αυξάνονται ταυτόχρονα. Όχι. Ο εξωτερικός i παίρνει μία τιμή και για αυτή την τιμή ολοκληρώνεται ολόκληρος ο εσωτερικός βρόχος. Πρώτα:

```
i = 1
```

και τότε ο j κάνει: j = 1 j = 2 j = 3 j = 4 Άρα έχουμε: (1,1) (1,2) (1,3) (1,4) Μετά τελειώνει ο εσωτερικός βρόχος. Ο i γίνεται:

```
i = 2
```

και ο j ξαναρχίζει από την αρχή: j = 1 j = 2 j = 3 j = 4 Άρα: (2,1) (2,2) (2,3) (2,4) Τέλος: 44 τότε το εσωτερικό σώμα εκτελείται: N × M φορές. Για παράδειγμα: N = 100 M = 50 σημαίνει: 100 × 50 = 5000 εκτελέσεις του εσωτερικού σώματος.

3.8.6 Το παράδειγμα της προπαίδειας

Στο το υλικό της Fortran υπάρχει ακριβώς αυτό το παράδειγμα:

```
program main
implicit none
integer :: a, i, j
do i = 2, 10
do j = 1, 10
a = j * i
write (*, '(i5)', advance='no') a
end do
print *
end do
end program main Εδώ έχουμε:
i = 2 ... 10
```

δηλαδή: 9 επαναλήψεις και: j = 1 ... 10 δηλαδή: 10 επαναλήψεις Άρα: 9 × 10 = 90 εκτελέσεις της εντολής: a = j * i Το ίδιο το υλικό το επισημαίνει: 9 × 10 = 90 επαναλήψεις. το υλικό της Fortran

3.8.7 Γιατί υπάρχει το `print *` μετά το εσωτερικό `DO` ;

Αυτό είναι πολύ ωραίο παράδειγμα για να καταλάβουμε και τη ροή. Μέσα στον εσωτερικό βρόχο έχουμε:

```
write (*,'(i5)',advance='no') a
```

Το: `advance='no'` σημαίνει ότι δεν αλλάζουμε γραμμή μετά την εκτύπωση. Η έξοδος θα ακολουθεί τη σειρά:

```
i = 1 j = 1  
i = 1 j = 2  
i = 2 j = 1  
i = 2 j = 2  
i = 3 j = 1  
i = 3 j = 2
```

Άρα έχουμε: $3 \times 2 = 6$ εκτελέσεις του `print` .

3.8.10 Τρία επίπεδα εμφώλευσης

Θεωρητικά μπορούμε να έχουμε:

```
do i = 1, 3  
do j = 1, 4  
do k = 1, 5
```

...

```
end do  
end do  
end do
```

Τότε το εσωτερικό σώμα εκτελείται: $3 \times 4 \times 5 = 60$ φορές. Δεν σημαίνει ότι πρέπει να χρησιμοποιούμε συνέχεια πολλά επίπεδα. Στην πράξη, τα δύο επίπεδα είναι εξαιρετικά συνηθισμένα.

3.8.11 Εμφωλευμένοι βρόχοι και πίνακες

Εδώ αρχίζει να γίνεται πραγματικά χρήσιμο. Αν έχουμε έναν πίνακα δύο διαστάσεων: Α με γραμμές και στήλες, μπορούμε να χρησιμοποιήσουμε:

```
do i = 1, n  
do j = 1, m  
print *, A(i,j)  
end do  
end do
```

Ο `i` μπορεί να αντιπροσωπεύει: γραμμή και ο `j` : στήλη Έτσι μπορούμε να περάσουμε από όλα τα στοιχεία ενός πίνακα. Οι διαστάσεις/εκτάσεις των συστοιχιών και η αναπαράσταση στοιχείων όπως `x(1,1)` , `x(1,2)` κ.λπ. εμφανίζονται στο βασικό υλικό της Fortran. το υλικό της Fortran

3.8.12 Ένας πίνακας 3×4

Φαντάσου: Αυτό είναι πολύ σημαντικό: Το απλό `EXIT` αφορά τον πλησιέστερο βρόχο στον οποίο βρίσκεται.

3.8.14 Πώς βγαίνουμε από εξωτερικό βρόχο;

Εδώ μας βοηθά ο ονομασμένος βρόχος που είδαμε προηγουμένως. `outer: do i = 1, 5`

```
do j = 1, 5  
if (condition) exit outer  
end do  
end do outer
```

Τώρα:

```
exit outer
```

σημαίνει: βγες από τον `outer` βρόχο. Το υλικό της Fortran παρουσιάζει ονομασμένες δομές `DO` , ακριβώς για τέτοιες περιπτώσεις. το υλικό της Fortran Για τις απλές ασκήσεις μας όμως, δεν χρειάζεται να το χρησιμοποιούμε αν δεν υπάρχει λόγος.

3.8.15 Και το `CYCLE` ;

Το ίδιο ισχύει και για το `CYCLE` . Σε:

```
do i = 1, 3  
do j = 1, 4  
if (j == 2) cycle  
print *, i, j  
end do  
end do
```

ΤΟ:

```
cycle
```

αφορά τον εσωτερικό DO . Άρα για κάθε i παραλείπεται το j=2 . Αποτέλεσμα: 1 1 1 3 1 4 2 1 2 3 2 4 3 1 3 3 3 4

3.8.16 Το πιο σημαντικό λάθος στους εμφωλευμένους

βρόχους Μην μπερδεύεις τους μετρητές.

3.9.2 IF ... ELSE μέσα σε DO

Μπορούμε φυσικά να έχουμε και ELSE .

```
do i = 1, 5
  if (mod(i, 2) == 0) then
    print *, i, "even"
  else
    print *, i, "odd"
  end if
end do
```

Αποτέλεσμα: 1 odd 2 even 3 odd 4 even 5 odd Άρα το IF εκτελείται μία φορά για κάθε επανάληψη του DO .

3.9.3 IF + DO WHILE

Το ίδιο μπορούμε να κάνουμε με DO WHILE .

```
i = 1
do while (i <= 10)
  if (mod(i, 2) == 0) then
    print *, i
  end if
  i = i + 1
end do
```

Εδώ έχουμε δύο διαφορετικές συνθήκες: Συνθήκη του DO WHILE $i \leq 10$ Αυτή αποφασίζει: Συνεχίζουμε τον βρόχο; Συνθήκη του IF $\text{mod}(i, 2) == 0$ Αυτή αποφασίζει: Τι εξετάζεται με τη συγκεκριμένη τιμή; Αυτός ο διαχωρισμός είναι πολύ σημαντικός.

3.9.4 IF + EXIT

Εδώ αρχίζει να γίνεται ακόμη πιο ενδιαφέρον. do

```
read *, x
if (x < 0) exit
print *, x
```

Έχουμε: DO ↓ διάβασε x

Άρα: Αυτό είναι ένα πολύ καλό παράδειγμα για να ξεχωρίζεις:

CYCLE → αγνόησε
EXIT → σταμάτα

3.9.7 IF μέσα σε εμφωλευμένα DO

Τώρα μπορούμε να συνδυάσουμε και τα τρία: εξωτερικό DO εσωτερικό DO IF Για παράδειγμα:

```
do i = 1, 3
  do j = 1, 4
    if (i == j) then
      print *, i, j
    end if
  end do
end do
```

Εδώ εξετάζουμε κάθε ζευγάρι (i,j) . Θα τυπωθούν: 1 1 2 2 3 3 Είναι ουσιαστικά ένας τρόπος να ψάχνουμε συγκεκριμένες θέσεις μέσα σε έναν δισδιάστατο χώρο. Αυτό θα μας φανεί πολύ χρήσιμο όταν περάσουμε στις συστοιχίες του Κεφαλαίου 4.

3.9.8 Παράδειγμα: εύρεση ενός αριθμού

Έστω ότι έχουμε:

```
integer :: i, x
do i = 1, 10
  read *, x
  if (x == 100) then
    print *, "Found!"
    exit
  end if
end do
```

Η λογική είναι: διάβασε ↓ είναι 100; ✓ ✗ ΝΑΙ ΟΧΙ ↓ ↓ FOUND επόμενη επανάληψη ↓

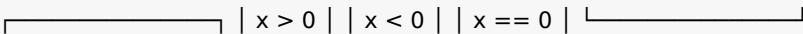
EXIT

Εδώ το EXIT μας επιτρέπει να σταματήσουμε νωρίτερα όταν βρούμε αυτό που ψάχνουμε.

3.9.14 Συνδυασμός με ELSE IF

Μπορούμε να έχουμε και πολλές κατηγορίες μέσα σε κάθε επανάληψη.

```
do i = 1, n
  read *, x
  if (x > 0) then
    print *, "positive"
  else if (x < 0) then
    print *, "negative"
  else
    print *, "zero"
  end if
end do
```

Εδώ: DO ↓ διάβασε x ↓  ↓ επόμενη επανάληψη Είναι

ακριβώς η λογική των IF / ELSE IF του Κεφαλαίου 2, αλλά τώρα επαναλαμβάνεται για πολλές τιμές.

3.9.15 Τι πρέπει να αναγνωρίζεις σε μια εκφώνηση

Από εδώ και πέρα, όταν διαβάζεις πρόβλημα, ψάξε δύο πράγματα: Α. Πόσες φορές επαναλαμβάνεται; Λέει: «Για τους 20 αριθμούς...» → γνωστό πλήθος → DO με μετρητή. Β. Τι απόφαση παίρνω για κάθε τιμή; Λέει: «Αν είναι θετικός...» → IF. Άρα: 20 αριθμοί ↓ DO ↓ για κάθε αριθμό ↓ IF. Γ. Πότε πρέπει να σταματήσω νωρίτερα; Λέει: «Μέχρι να βρεθεί το 0.» → EXIT. Στο υλικό του Κεφαλαίου 1 η διαδικασία είναι να εντοπίζουμε πρώτα δεδομένα και ζητούμενα, να γράφουμε τους υπολογισμούς και μετά να τους μετατρέπουμε σε Fortran.

Κεφάλαιο_1_Εισαγωγή_στη_Fortran Στο Κεφάλαιο 3 προσθέτουμε

ένα ακόμη ερώτημα: Πόσες φορές πρέπει να επαναληφθεί η διαδικασία;

3.10.2 Πρώτη ερώτηση: Ξέρω πόσες φορές;

Αυτό είναι το πρώτο φίλτρο. Περίπτωση Α — Ναι Η εκφώνηση λέει: «Διάβασε 10 αριθμούς.» Ξέρουμε: N = 10 Άρα φυσικά:

```
do i = 1, 10
```

...

```
end do
```

Περίπτωση Β — Όχι Η εκφώνηση λέει: «Διάβαζε αριθμούς μέχρι να δοθεί αρνητικός αριθμός.» Δεν ξέρουμε πόσοι αριθμοί θα δοθούν. Μπορεί να είναι: ή: ή: Άρα δεν έχουμε φυσικό:

```
do i = 1, N
```

Χρειαζόμαστε επανάληψη που ελέγχεται από συνθήκη.

3.10.3 Περίπτωση 1: Γνωστό πλήθος επαναλήψεων

Αν η εκφώνηση περιέχει λέξεις όπως: «10 φορές» «για τους αριθμούς 1 έως 100» «διάβασε 20 τιμές» «για κάθε i από 1 έως n» τότε έχουμε γνωστό πλήθος. Παράδειγμα: Να υπολογιστεί το άθροισμα των ακεραίων από 1 έως 100. Δεν χρειάζεται να ψάξουμε πότε θα σταματήσουμε. Ξέρουμε: 1, 2, 3, ..., 100 Άρα:

```
sum = 0
do i = 1, 100
  sum = sum + i
end do
```

3.10.4 Η λογική πίσω από το πρόγραμμα

Η δεύτερη μορφή είναι συχνά πιο καθαρή όταν η τιμή που τερματίζει πρέπει πρώτα να διαβαστεί. Το ίδιο μοτίβο υπάρχει και στο υλικό του καθηγητή: ζητείται επαναληπτική είσοδος μέχρι το άθροισμα να φτάσει/ξεπεράσει το 100, με λογική «όσο... επανάλαβε». το υλικό της Fortran

3.10.7 Πολύ σημαντικό: η «ειδική τιμή»

Στις ασκήσεις συχνά υπάρχει μια τιμή που δεν αποτελεί κανονικό δεδομένο, αλλά σημαίνει: «Σταμάτα.» Για παράδειγμα: -1 ή: ή κάποιος άλλος αριθμός. Αυτή λέγεται συχνά τιμή-φρουρός (sentinel). Παράδειγμα: Διάβαζε βαθμούς μέχρι να δοθεί -1. Τότε: -1 → δεν είναι βαθμός -1 → σημαίνει ΤΕΛΟΣ Άρα πρέπει να προσέχουμε να μην τον συμπεριλάβουμε στους υπολογισμούς, εκτός αν η εκφώνηση λέει διαφορετικά.

3.10.8 Παράδειγμα με άθροισμα

Εκφώνηση: Να διαβάζονται θετικοί ακέραιοι και να υπολογίζεται το άθροισμά τους. Η διαδικασία σταματά όταν δοθεί αρνητικός αριθμός. Σκεφτόμαστε: 1. Δεν ξέρω πόσοι αριθμοί. → όχι απλό DO $i=1,N$. 2. Υπάρχει συνθήκη τερματισμού. → αρνητικός αριθμός. 3. Πρέπει να προσθέτω κάθε κανονική τιμή. → $sum = sum + x$. Άρα:

```
sum = 0
```

do

```
read *, x
if (x < 0) exit
sum = sum + x
end do
print *, sum
```

Αυτό είναι εκφώνηση → λογική → κώδικας.

3.10.9 Τρίτη περίπτωση: σταματάμε όταν συμβεί κάτι

Δεν υπάρχει πάντα sentinel. πρέπει να ξεκινήσουμε από:

```
fact = 1
```

και όχι:

```
fact = 0
```

Γιατί: $0 \times \text{οτιδήποτε} = 0$ οπότε θα καταστρέφαμε όλο τον υπολογισμό. Κανόνας Πριν από τον βρόχο, αρχικοποίησε τη μεταβλητή με την ουδέτερη τιμή της πράξης. Πράξη Αρχική τιμή Άθροισμα 0 Γινόμενο 1 Μετρητής ανάλογα με το πρόβλημα

3.11.2 Ξεχνάω να ενημερώσω τη συνθήκη

Αυτό είναι από τα πιο επικίνδυνα λάθη.

```
i = 1
do while (i <= 10)
print *, i
end do
```

Το i παραμένει: Άρα: $1 \leq 10$ παραμένει .TRUE. για πάντα. Διόρθωση

```
i = 1
do while (i <= 10)
print *, i
i = i + 1
end do
```

Τώρα: $1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow 10 \rightarrow 11$ και τελικά: $11 \leq 10 \rightarrow$.FALSE.

3.11.3 Αλλάζω τη μεταβλητή προς λάθος κατεύθυνση

Ακόμη κι αν την αλλάξεις, μπορεί να την αλλάξεις λάθος.

```
i = 1
do while (i <= 10)
i = i - 1
end do
```

Έχουμε:

Άρα:

```
i = 5
```

→ CYCLE → $i = 5 \rightarrow$ CYCLE → $i = 5$ Άπειρος βρόχος. →

3.11.8 Πώς το διορθώνουμε;

Μία λύση:

```
i = 1
do while (i <= 10)
if (i == 5) then
i = i + 1
cycle
end if
print *, i
i = i + 1
end do
```

Τώρα όταν $i=5$: ↓

```
i = 6
```


↓

CYCLE

↓ επόμενη επανάληψη Άρα δεν κολλάμε.

3.11.9 Άλλο λάθος: EXIT που δεν μπορεί ποτέ να

συμβεί do $x = x + 1$

```
if (x < 0) exit  
end do
```

Αν το x ξεκινά από: και συνεχώς αυξάνεται: $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \dots$ δεν πρόκειται ποτέ να γίνει: $x < 0$ Άρα το EXIT υπάρχει στον κώδικα, αλλά λογικά δεν είναι προσβάσιμο. Πρέπει να ελέγχεις όχι μόνο: «Έχω EXIT ;» αλλά: «Μπορεί πράγματι να ενεργοποιηθεί;»

3.12 Ασκήσεις εφαρμογής και έλεγχος γνώσεων

Οι ασκήσεις που ακολουθούν προορίζονται για αυτοτελή εξάσκηση. Δεν δίνονται λύσεις σε αυτή την έκδοση, ώστε να μπορεί να ελεγχθεί πραγματικά η κατανόηση των δομών επανάληψης.

3.12.1 Αριθμημένο DO

Να εμφανιστούν οι ακέραιοι από 1 έως 20 και στη συνέχεια μόνο οι άρτιοι.

3.12.2 Αρχή, τέλος και βήμα

Να εμφανιστούν οι αριθμοί 2, 5, 8, 11, ... μέχρι το 50. Να προσδιοριστεί ποια είναι η τελευταία τιμή που πραγματικά εμφανίζεται.

3.12.3 Αντίστροφη μέτρηση

Να εμφανιστούν οι ακέραιοι από 20 έως 1 χρησιμοποιώντας αρνητικό βήμα.

3.12.4 Άθροισμα

Να διαβάζεται θετικός ακέραιος N και να υπολογίζεται το άθροισμα $1+2+\dots+N$.

3.12.5 Παραγοντικό

Να διαβάζεται θετικός ακέραιος N και να υπολογίζεται το $N!$ με επανάληψη.

3.12.6 Άθροισμα και γινόμενο

Για δεδομένο N να υπολογίζονται στον ίδιο βρόχο το άθροισμα και το γινόμενο των ακεραίων $1..N$.

3.12.7 Ενδιάμεσες τιμές

Να υπολογίζεται $S=1+1/2+1/3+\dots+1/N$ και να εμφανίζεται η τιμή του S μετά από κάθε όρο.

3.12.8 Σειρά

Να υπολογιστεί η σειρά $S=5+3-9+27-81+\dots\pm 3^N$ και να εμφανίζονται οι ενδιάμεσες και η τελική τιμή.

3.12.9 DO WHILE

Να διαβάζονται ακέραιοι μέχρι να δοθεί 0 και να υπολογίζεται το άθροισμα των τιμών που προηγήθηκαν.

3.12.10 DO + EXIT

Να διαβάζονται θετικοί ακέραιοι και να προστίθενται. Η επανάληψη να τερματίζει όταν το άθροισμα ξεπεράσει το 100.

3.12.11 CYCLE

Να εμφανιστούν οι ακέραιοι $1..100$ εκτός από όσους διαιρούνται ακριβώς με το 5. Να χρησιμοποιηθεί CYCLE.

3.12.12 Εμφωλευμένοι βρόχοι

Να εμφανιστούν όλα τα ζεύγη (i,j) με $i=1..5$ και $j=1..5$. Στη συνέχεια να εμφανιστούν μόνο όσα ικανοποιούν $i < j$.

3.12.13 Διπλασιασμός ποσού

Αρχικό ποσό 0.01 €. Να διπλασιάζεται καθημερινά μέχρι το συνολικό ποσό να ξεπεράσει τα 100.000.000 €. Να εμφανίζονται η ημέρα, το ποσό της ημέρας και το συνολικό ποσό.

3.12.14 IF + DO

Να διαβάζεται N και να υπολογίζεται το άθροισμα μόνο των περιττών αριθμών $1..N$, με DO και IF. Checklist Κεφαλαίου 3 ☐ Να εξηγείται τι είναι βρόχος και γιατί χρησιμοποιείται. ☐ Να γράφεται αριθμημένο DO με αρχή, τέλος και βήμα. ☐ Να αναγνωρίζονται θετικό και αρνητικό βήμα και τα ασυμβατά όρια. ☐ Να υπολογίζεται το πλήθος επαναλήψεων ενός αριθμημένου DO. ☐ Να χρησιμοποιείται συσσωρευτής για άθροισμα και γινόμενο. ☐ Να εμφανίζονται ενδιάμεσα αποτελέσματα όταν ζητούνται. ☐ Να γράφεται DO WHILE και να ελέγχεται η συνθήκη έναρξης/τερματισμού.

☐ Να χρησιμοποιείται DO + IF + EXIT όταν το κριτήριο τερματισμού βρίσκεται μέσα στο σώμα. ☐ Να ξεχωρίζει το EXIT από το CYCLE. ☐ Να αναγνωρίζεται ο κίνδυνος άπειρου βρόχου όταν η μεταβλητή της συνθήκης δεν οδηγείται προς τον τερματισμό. ☐ Να γράφονται εμφωλευμένοι βρόχοι και να παρακολουθούνται σωστά οι μετρητές. ☐ Να συνδυάζονται IF και DO. ☐ Να μετατρέπεται μια εκφώνηση σε: δεδομένα $\rightarrow$ ζητούμενα $\rightarrow$ επανάληψη $\rightarrow$ αρχικοποίηση $\rightarrow$ σώμα $\rightarrow$ ενημέρωση $\rightarrow$ τερματισμός.